任务 1：数据库与查询条件基础说明

页面中的输入为什么会与数据库查询有关

在许多网页应用中，为了实现动态内容的展示和交互功能，需要从数据库中获取数据。例如，一个电商网站的商品展示页面，需要根据用户选择的分类、价格区间等条件，从数据库中筛选出符合要求的商品信息并展示给用户。这就需要将页面中的用户输入作为查询条件，传递给数据库进行查询操作，以获取相应的数据。

一个最小业务链路是什么

以一个简单的用户登录业务为例，最小业务链路为：输入用户名和密码（输入条件）→ 后端代码将输入的用户名和密码拼接成 SQL 查询语句，向数据库发起查询（数据库查询）→ 数据库根据查询语句返回匹配的用户信息（返回结果）。如果返回的用户信息存在且密码匹配，则登录成功；否则登录失败。

用户输入是如何进入查询条件的

在未进行安全防护的代码中，后端程序通常会直接将用户输入的字符串拼接进 SQL 查询语句中。例如，在 PHP 代码中，可能会有如下操作：

```php
$username = $_POST['username'];
$password = $_POST['password'];
$sql = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
$result = mysqli_query($conn, $sql);
```

这里，用户输入的 $username 和 $password 直接被拼接进了 SQL 语句中，成为了查询条件的一部分。

为什么这为 SQL 注入创造了可能性

由于用户输入直接进入了查询条件，攻击者可以通过精心构造特殊的输入字符串，改变原本 SQL 查询语句的结构和逻辑。例如，攻击者在用户名输入框中输入 ' OR '1'='1，密码输入框中输入任意值，此时拼接后的 SQL 语句变为：

```sql
SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '任意值'
```

由于 '1'='1 永远为真，这个查询语句就会返回 users 表中的所有记录，从而绕过了正常的登录验证，实现了 SQL 注入攻击。

任务 2：DVWA SQL Injection Low 现象记录

实验背景

本实验基于 DVWA 平台进行。DVWA 是一个专门用于安全测试的脆弱 Web 应用，它允许我们设置不同的安全级别（低、中、高、不可能），从而直观地观察 SQL 注入漏洞的产生原因及相应的防御机制。

实验现象分析

1.低安全级别下的注入攻击

现象：在 User ID 输入框中输入 1' or '1'='1 并提交后，页面返回了数据库中所有的用户信息（如 admin、Gordon Brown、Hack Me 等）。

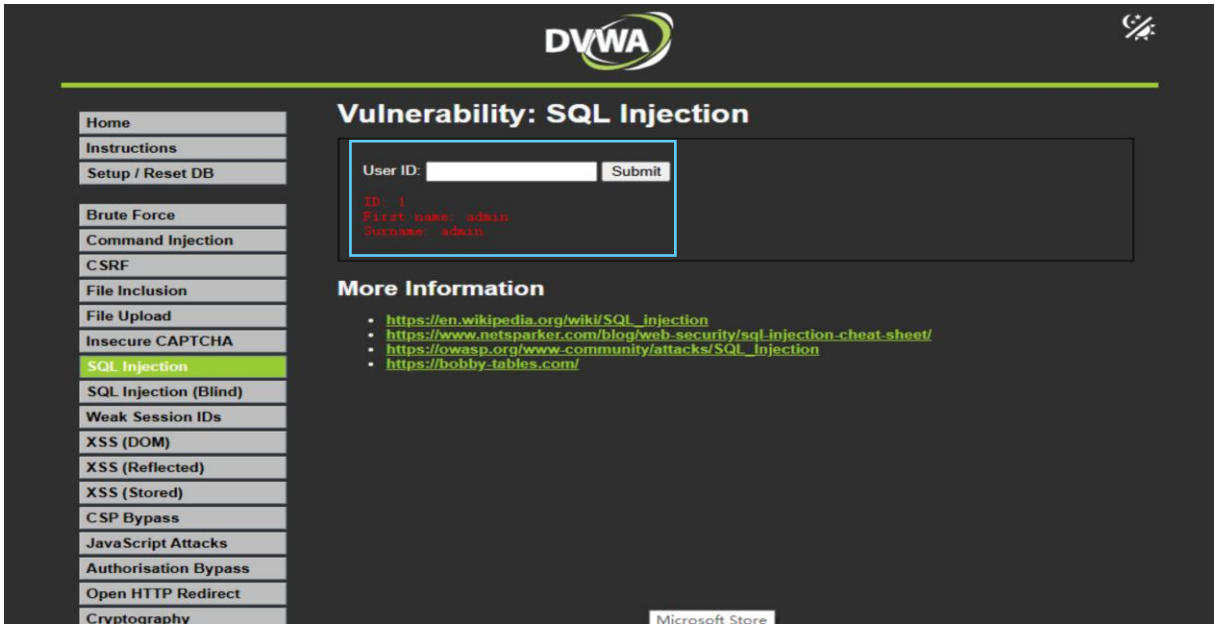
原理：在低安全级别下，后端代码未对用户输入进行任何过滤或转义。输入的字符串直接拼接到了 SQL 查询语句中。原本的查询逻辑可能是 SELECT * FROM users WHERE id = '$id'。当输入 1' or '1'='1 时，SQL 语句变成了：

sql 编辑

```sql
SELECT * FROM users WHERE id = '1' or '1'='1'
```

结果：由于 '1'='1' 永远为真，这就导致 WHERE 子句恒成立，数据库因此返回了表中的所有记录。这演示了最基础的 SQL 注入原理。

2.正常查询与防御对比



现象：在 User ID 输入框中输入 1 并提交，页面仅返回了 ID 为 1 的用户信息（admin）。

对比：这展示了应用程序在正常情况下的预期行为。与图 1 的攻击结果相比，可以清晰地看到，如果输入未经恶意构造，系统只会返回单条匹配记录。这为理解注入攻击的“异常”提供了基准。

3.安全级别设置

功能：此页面是 DVWA 的核心控制面板，用于设置整个应用的安全级别。图中显示当前级别为 Low。

级别说明：

Low：完全脆弱，无任何安全措施，用于演示最基本的漏洞。

Medium：采用了较差的安全实践，可能使用了简单的过滤函数（如 mysql_real_escape_string）或前端限制，但容易被绕过。

High：采用了更复杂的防御手段，如更严格的输入验证或限制查询次数，增加了攻击难度。

Impossible：采用了最安全的编码实践，通常是使用预编译语句（Prepared Statements）和参数化查询，从根本上杜绝了 SQL 注入的可能性。

任务总结与思考

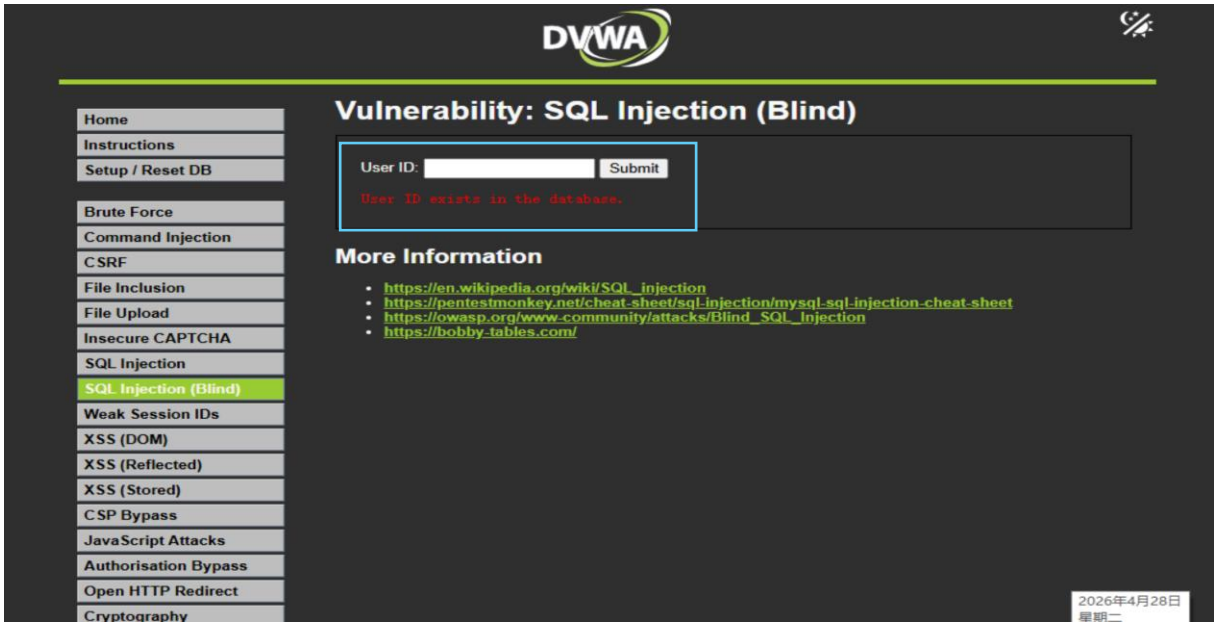
通过这个实验，我们可以得出以下结论：

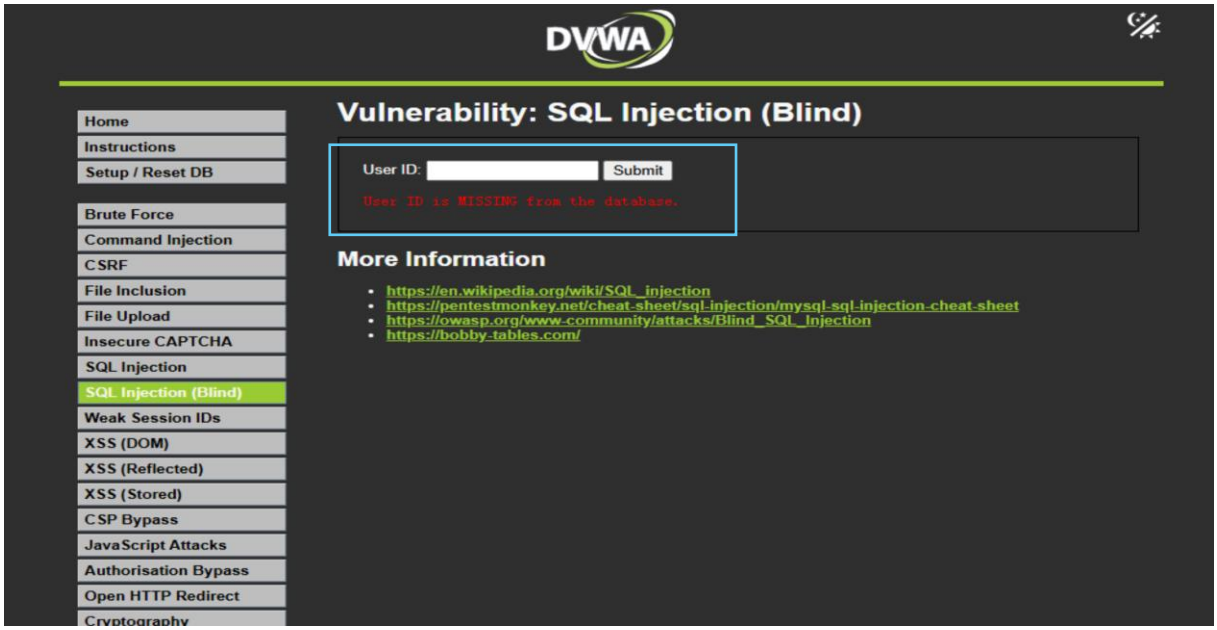
- **漏洞根源**：SQL 注入的根本原因在于程序将用户的输入数据当作代码的一部分来执行。
- **防御层级**：防御 SQL 注入是一个渐进的过程。从低级别的无防御，到中高级别的输入过滤和验证，再到最高级别的预编译语句，安全性逐级提升。
- **最佳实践**：要彻底防止 SQL 注入，最有效的方法是使用参数化查询（预编译语句），因为它能确保用户输入的数据和 SQL 代码被严格区分开，从根本上切断了攻击路径。

任务 3：机制分析与差异比较

输入如何进入查询结构（输入—拼接—解析—执行—回显或行为差异链条）

从提供的图片中可以看到，DVWA（DamVulnerable Web Application）中的 SQL 注入漏洞示例清晰地展示了输入进入查询结构的过程。





输入：用户在“User ID”输入框中输入特定的值，例如在普通 SQL 注入示例中输入

1，在盲注示例中可能输入一些构造的 payload 如 1' AND 1=1 -- 等。

拼接：后端代码将用户输入的值直接拼接到 SQL 查询语句中。例如，原本的 SQL 查询语句可能是 SELECT * FROM users WHERE user_id = [用户输入]，当用户输入 1 时，拼接后的语句变为 SELECT * FROM users WHERE user_id = 1；若用户输入恶意 payload，如 1' OR '1'='1 -- ，拼接后语句变为 SELECT * FROM users WHERE user_id = '1' OR '1'='1 -- '。

解析：数据库服务器对拼接后的 SQL 语句进行解析，理解其中的语义和操作要求。

执行：数据库服务器执行解析后的 SQL 语句，进行相应的数据查询操作。

回显或行为差异：在普通 SQL 注入中，如图片所示，当输入合法的 User ID（如 1）时，页面会回显对应的用户信息，包括 ID、First name、Surname 等；而当输入存在注入攻击时，可能会回显更多的数据或者不同的结果。在盲注情况下，页面不会直接回显数据库查询的具体数据内容，而是通过一些行为差异，如页面响应时间的变化、页面显示内容是否存在等，来让攻击者推断数据库中的信息。

为什么根因是“结构被污染”而不是“字符本身危险”

从上述过程可以看出，像单引号'、OR、AND 等字符本身并不具备危险性。例如，在正常的数据库操作中，单引号用于界定字符串，OR 和 AND 是逻辑运算符，它们都是 SQL 语言的合法组成部分。

然而，当用户输入被直接拼接到查询结构中时，这些字符可能会改变原本查询语句的结构和逻辑。例如，输入 1' OR '1'='1 -- ，其中的单引号闭合了原本的字符串界定，OR 条件使得查询逻辑变为只要'1'='1'这个条件成立（显然总是成立），就会返回结

果，从而绕过了原本基于特定 user_id 的查询限制。所以，真正危险的不是字符本身，而是用户输入进入了查询结构，导致查询结构被污染，改变了原本的查询逻辑。

Low 与 Medium 的差异在哪里

- **Low 级别**：从图片中可以看出，在 Low 安全级别下，DVWA 几乎没有对用户输入进行任何有效的过滤或处理。用户输入可以直接拼接到 SQL 查询语句中，这使得攻击者能够非常容易地构造各种 SQL 注入 payload，如简单的 1'

OR '1'='1 -- 就能获取到数据库中的大量信息，如用户列表等。

- **Medium 级别**：在 Medium 安全级别下，DVWA 对用户输入进行了一定程度的处理，例如对一些特殊字符进行了转义等操作。这使得一些简单的、直接的 SQL 注入攻击 payload 可能无法生效，增加了攻击的难度。但这种处理并不完善，攻击者仍然可以通过一些技巧绕过这些过滤措施，实现 SQL 注入攻击。

普通 SQL 注入与 Blind SQL Injection 的差异在哪里

普通 SQL 注入：如图片中所示，在普通 SQL 注入中，攻击者能够直接从页面的回显结果中获取到数据库中的具体信息。例如，当输入特定的 User ID 或注入 payload 后，页面会直接显示与之对应的用户信息，攻击者可以直观地看到查询结果，从而快速获取数据库中的数据。

Blind SQL Injection（盲注）：在盲注情况下，页面不会直接显示数据库查询的具体结果。攻击者需要通过观察页面的一些细微变化，如响应时间、页面是否显示特定内

容等，来推断数据库中的信息。例如，攻击者可能构造带有时间延迟函数的 payload，如 1' AND IF(1=1,SLEEP(5),0) -- ，如果页面响应时间明显变长，说明条件 1=1 为真，攻击者可以据此逐步猜测数据库中的用户名、密码等信息。

为什么盲注和普通注入根因相同，但证据形式不同

盲注和普通注入的根因都是用户输入进入了查询结构，导致查询结构被污染，改变了原本的查询逻辑。然而，由于应用程序对查询结果的处理方式不同，导致了证据形式的差异。

在普通注入中，应用程序直接将数据库查询的结果返回给用户，所以攻击者可以直接从页面回显中获取到数据，证据形式就是页面直接显示的数据内容。

而在盲注中，应用程序对查询结果进行了某种处理，例如只返回固定的成功或失败信息，或者不返回任何与查询结果直接相关的信息。攻击者无法直接获取数据，只能通过观察应用程序的其他反应，如响应时间、页面状态等，来推断数据库中的信息，所以证据形式表现为这些间接的行为差异。

强制判断题

1. **SQL 注入的根因更接近**
    - **答案**：B. 输入进入了不该进入的查询结构
    - **理由**：如前面分析，字符本身并不危险，危险在于输入进入了查询结构，改变了查询逻辑，导致 SQL 注入漏洞的产生。
2. **Blind SQL Injection 和普通 SQL 注入的最大差异更接近**
    - **答案**：B. 证据形式不同

- **理由**：两者根因相同，都是查询结构被污染，但普通注入能直接从页面回显获取信息，盲注需要通过观察页面行为差异推断信息，所以最大差异在于证据形式不同。

你最容易犯的错误及改法

- **错误**：把 payload 当根因。例如，只关注输入的 1' OR '1'='1 -- 这样的字符串，认为这些字符本身就是导致 SQL 注入的原因。
- **原因**：只盯住字符串表面，没有看到“查询结构”这一层，没有理解到是输入进入查询结构改变了逻辑才导致漏洞。
- **改法**：每次分析都回到一句话：问题发生在输入内容本身，还是发生在输入被拼进了查询结构里？通过这种方式，深入理解 SQL 注入的本质是查询结构被污染，而不是特定的字符组合。

任务 4：SQL 注入防护清单与解释

参数化查询

它解决什么问题：参数化查询将用户输入作为参数传递给 SQL 语句，而不是直接拼接进语句中。这样可以确保用户输入被当作数据而不是代码的一部分处理，有效防止攻击者通过构造特殊输入改变查询语句的结构和逻辑，从而避免 SQL 注入攻击。

它控制哪一层：参数化查询控制的是根因层，从根本上解决了用户输入污染查询结构的问题。

它不能替代什么：参数化查询不能替代输入校验。虽然参数化查询可以防止 SQL 注入，但对于一些明显不符合业务规则的输入（如长度过长、格式错误等），仍然需要通

过输入校验进行过滤，以保证数据的合法性和应用程序的稳定性。

输入校验

它解决什么问题：输入校验对用户输入进行严格的检查和过滤，确保输入符合预期的格式、类型和范围等要求。例如，对于数字类型的输入，只允许输入数字字符；对于字符串类型的输入，限制其长度等。通过输入校验可以防止一些非法输入进入后续的处理流程，减少 SQL 注入等攻击的风险。

它控制哪一层：输入校验控制的是暴露面层，通过限制用户输入的范围和格式，减少了攻击者可以利用的输入机会，降低了系统暴露在攻击下的风险。

它不能替代什么：输入校验不能替代参数化查询。输入校验只能对输入进行初步的过滤，但对于一些经过精心构造的恶意输入，可能仍然无法完全防范。参数化查询从查询结构的角度进行防护，是更根本的解决方案，二者需要结合使用。

错误处理

它解决什么问题：错误处理对数据库操作过程中可能出现的错误进行合理的处理，避免将详细的错误信息直接返回给用户。例如，当数据库查询出现错误时，不返回具体的 SQL 错误信息，而是返回一个通用的错误提示。这样可以防止攻击者通过分析错误信息获取数据库的结构、表名、字段名等敏感信息，从而进行更有针对性的攻击。

它控制哪一层：错误处理控制的是后果范围层，通过限制错误信息的暴露，减少了攻击者利用错误信息进行攻击的可能性，降低了攻击造成的后果严重程度。

它不能替代什么：错误处理不能替代参数化查询和输入校验。错误处理只是在攻击发生后对后果进行一定程度的控制，而不能从根本上防止 SQL 注入攻击的发生。参数

化查询和输入校验是从源头上预防攻击的措施，是更重要的防护手段。

最小权限

它解决什么问题：最小权限原则要求数据库用户只拥有完成其工作所必需的最小权限。例如，一个只负责查询数据的应用程序账号，不应该拥有删除、修改数据的权限。这样可以限制攻击者在成功利用 SQL 注入漏洞后能够进行的操作，即使攻击者能够注入恶意 SQL 语句，也只能在有限的权限范围内执行，减少了攻击造成的损失。

它控制哪一层：最小权限控制的是后果范围层，通过限制数据库用户的权限，降低了攻击者在成功攻击后能够造成的危害程度。

它不能替代什么：最小权限不能替代参数化查询、输入校验和错误处理。最小权限是在攻击发生后对后果进行控制，而参数化查询、输入校验和错误处理是从不同角度预防攻击的发生，它们相互配合，共同构建完整的 SQL 注入防护体系。

其他合理措施：使用 Web 应用防火墙（WAF）

它解决什么问题：WAF 可以对进入 Web 应用的 HTTP 请求进行实时监测和过滤，识别并拦截常见的攻击行为，包括 SQL 注入攻击。WAF 通常具有预定义的规则库，能够检测和阻止各种已知的攻击模式，为 Web 应用提供额外的安全防护层。

它控制哪一层：WAF 控制的是暴露面层，在网络层面拦截攻击请求，减少攻击到达 Web 应用服务器的机会。

它不能替代什么：WAF 不能替代应用程序内部的安全措施，如参数化查询、输入校验等。WAF 只能作为一道外部防线，对于一些经过巧妙构造能够绕过 WAF 规则的

攻击，仍然需要应用程序内部的安全机制进行防护。

收口项 1：SQL 注入因果链/防护关系图

mermaid

```mermaid
graph LR
    A[输入点] --> B[查询构造]
    B --> C{查询结构是否被污染?}
    C -->|是| D[数据库解析/执行恶意查询]
    C -->|否| E[数据库解析/执行正常查询]
    D --> F[结果差异或行为证据 - 返回大量数据/异常响应]
    E --> G[结果差异或行为证据 - 返回正常数据]
    H[根因判断 - 结构被污染] --> C
    I[防护控制点1 - 参数化查询] --> B
    J[防护控制点2 - 最小权限] --> D
```

关系图说明

(1)输入点的用户输入进入查询构造环节，如果查询结构被污染（如用户输入未经处理直接拼接进查询语句），数据库会解析并执行恶意查询，导致结果出现异常，如返回大量数据或页面响应异常等，根因在于查询结构被污染。

(2)参数化查询防护控制点作用于查询构造环节，确保用户输入作为参数传递，不污染查询结构；最小权限防护控制点作用于数据库执行恶意查询环节，限制攻击者利用漏洞后的操作权限，减少危害。

收口项 2：学生自评表

1.我本章最清楚的一点是什么：我最清楚 SQL 注入的根因是查询结构被污染，而不是某些特殊字符本身危险。通过任务分析和实验，我理解了用户输入如何进入查询结构并改变其逻辑，以及不同防护措施如何从不同层面解决这个问题。

2.我最容易混淆的一点是什么：我最容易混淆普通 SQL 注入和盲注的证据形式差异。一开始不太理解为什么盲注不能直接从页面返回结果获取信息，通过进一步学习

和分析，才明白是由于应用程序对查询结果的处理方式不同导致证据形式不同。

3.我本次作业中哪一段最需要教师复核：我认为任务 3 中关于 Low 与 Medium 差异以及普通 SQL 注入与盲注差异的分析部分最需要教师复核。这部分内容涉及对不同级别和不同类型注入的理解和分析，我担心自己的理解不够准确和全面。

4.如果重做一次，我会优先改哪一部分：如果重做一次，我会优先改进任务 2 中对实验现象的记录和分析部分。我会更加详细地描述普通输入和异常输入的差异以及返回结果的不同，并且更深入地分析这些差异支撑的判断，使实验记录更加完整和准确。

收口项 3：AI 输出审核与人工复核记录

本作业哪些内容使用过 AI 辅助：在撰写任务 3 中关于机制分析与差异比较的部分内容时，我使用了 AI 辅助生成一些基本的分析思路和要点框架，例如对 Low 与 Medium 差异、普通 SQL 注入与盲注差异的初步描述。

哪些判断是你自己复核后保留的：在对 AI 生成的内容进行复核后，我保留了关于 SQL 注入根因是结构被污染、Low 与 Medium 在输入处理方式上的差异、普通 SQL 注入与盲注证据形式不同等关键判断。这些判断与我在课程学习和实验过程中的理解相符，并且有相应的理论和实践依据支持。

哪些内容你发现 AI 输出不准确并进行了修正：我发现 AI 输出在描述盲注的工作原理时不够准确和详细。AI 只是简单提到了盲注通过观察页面变化获取信息，但没有具体说明是基于什么变化（如响应时间、页面内容状态等）以及如何逐步推断数据库信息。我对此进行了修正，补充了基于时间的盲注和基于布尔的盲注的具体工作原理和示例。

你最终如何确认结论和证据是一一对应的：在完成作业过程中，我对于每一个结论都仔细回顾了相应的实验现象、代码分析以及理论知识。例如，在得出 SQL 注入根因是结构被污染的结论时，我结合了 DVWA 实验中普通输入和异常输入导致不同返回结果的现象，以及代码中直接拼接用户输入导致查询结构改变的分析；在分析防护措施时，针对每个措施都说明了其解决的问题和控制的层面，并与 SQL 注入的根因和攻击过程相对应，确保结论和证据紧密关联、一一对应。